

Software Effort Estimation

Research Demo — Technical Documentation

FAHP-Weighted Kernel SVR with COCOMO-81 & NASA-93 Datasets

Keshav · Mitacs Internship Proposal Demo

Built with Streamlit · Sehra et al. (2018) methodology

Dataset: PROMISE Repository (NASA-93, 93 projects, 1971–1987)

Table of Contents

- 1. Project Overview**
- 2. Datasets**
 - 2.1 COCOMO-81 Dataset
 - 2.2 NASA-93 Dataset (PROMISE Repository)
- 3. Methodology**
 - 3.1 COCOMO-81 Baseline Model
 - 3.2 Chang (1996) FAHP Feature Weighting
 - 3.3 FAHP Pairwise Comparison Matrix
 - 3.4 Weighted-Kernel SVR (approximating LSSVM)
 - 3.5 Leave-One-Out Cross-Validation (LOO-CV)
 - 3.6 MMRE — Mean Magnitude of Relative Error
 - 3.7 Empirical 80% Prediction Interval
- 4. Training Pipeline**
- 5. Application Architecture**
 - 5.1 Streamlit App (app.py)
 - 5.2 Model Training (train_model.py)
 - 5.3 File Structure
- 6. App User Guide**
 - 6.1 Sidebar — Project Inputs
 - 6.2 Pipeline Steps Panel
 - 6.3 Estimate Button & Results
 - 6.4 SHAP Explainability Chart
 - 6.5 Confidence Band Chart
 - 6.6 What-If Optimizer
 - 6.7 GenAI Productivity Adjustment
 - 6.8 Validity & Limitations Panel
- 7. Known Issues & Honest Limitations**
- 8. Code Bugs Fixed**
- 9. References**

1. Project Overview

This project is a **Streamlit-based interactive research demo** for explainable software effort estimation, built as supporting material for a Mitacs Globalink internship proposal. It implements and extends the methodology of Sehra, Brar, Kaur & Kaur (2018), which combines Fuzzy Analytic Hierarchy Process (FAHP) feature weighting with a Least Squares Support Vector Machine (LSSVM) regressor trained on historical software project data.

The primary research question is: *can explainable AI techniques (SHAP, FAHP weights, prediction intervals) make black-box SVR-based effort estimates more transparent and trustworthy for project managers?*

What the demo does

- Accepts 16 COCOMO-81 cost driver inputs (LOC, 15 effort multipliers) from the user.
- Applies Chang (1996) FAHP extent analysis to derive feature importance weights from the training data.
- Trains a weighted-kernel RBF SVR that embeds the FAHP weights directly into the kernel function.
- Predicts effort in person-months (PM) with an empirical 80% prediction interval derived from LOO signed errors.
- Explains the prediction using SHAP KernelExplainer — showing each feature's PM-unit contribution.
- Shows a confidence band chart plotting LOO predictions vs. actuals across all 156 training projects.
- Provides a What-If Optimizer suggesting which cost driver changes would most reduce estimated effort.
- Applies a post-hoc GenAI productivity adjustment (18–45%) sourced from Peng et al. 2023 and McKinsey 2023.
- Classifies project risk (Low / Medium / High) using EAF, LOC, and predicted effort.

Model type	RBF SVR (sklearn) approximating LSSVM
Feature weighting	Chang (1996) FAHP extent analysis via Spearman correlation proxy
Training data	156 projects: COCOMO-81 (63) + NASA-93 PROMISE (93)
Evaluation	Leave-One-Out Cross-Validation (LOO-MMRE)
Explainability	SHAP KernelExplainer (background=30, nsamples=150)
PI method	Empirical 80% from 10th/90th percentile of LOO signed relative errors
Frontend	Streamlit with custom CSS (dark sidebar, white content cards)
Python deps	scikit-learn, scipy, pandas, numpy, shap, streamlit, matplotlib

2. Datasets

2.1 COCOMO-81 Dataset

The COCOMO-81 dataset was collected by Barry Boehm (1981) for the original Constructive Cost Model. It contains 63 software projects spanning a wide range of application domains including business, scientific, systems, and real-time software. Each project is described by 16 features: lines of code (KLOC) and 15 cost driver multipliers.

Projects	63
Source	Boehm, B. (1981). Software Engineering Economics. Prentice-Hall.
File	cocomo81.csv (loaded at training time)
Features	loc, rely, data, cplx, time, stor, virt, turn, acap, aexp, pcap, vexp, lexp, modp, tool, sced
Target	actual — effort in person-months (PM)
LOC range	Approx. 1 – 1000 KLOC
Effort range	Approx. 6 – 11,400 PM

2.2 NASA-93 Dataset (PROMISE Repository)

The NASA-93 dataset is part of the PROMISE Software Engineering Repository and contains 93 NASA software projects from 1971 to 1987. It was donated by Tim Menzies and the data was originally collected by Jairus Hihn at the Jet Propulsion Laboratory (JPL/NASA). Projects include both flight and ground systems across 6 NASA centers.

The dataset uses the same COCOMO-81 feature schema (16 features + actual effort). The original ARFF file encodes cost drivers as categorical ratings (v/l/n/h/vh/xh — very low to extra high). For integration with the SVR pipeline these ratings are converted to the standard Boehm (1981) Table 8-1 multiplier values before training.

Projects	93
Source	PROMISE Software Engineering Repository
ARFF location	timm/ourmine, our/arffs/effest/nasa93.arff
Donated by	Tim Menzies (West Virginia University)
Data from	Jairus Hihn, Jet Propulsion Laboratory / NASA
Years	1971 – 1987
Domain	Flight systems, ground systems, avionics, mission planning, OS, data capture
Centers	6 NASA centers (including JPL, Goddard, Johnson)
LOC range	0.90 – 980.00 KLOC

Effort range8.4 – 8211.0 PM

Rating encodingv/l/n/h/vh/xh → COCOMO-81 multipliers per Boehm (1981) Table 8-1

COCOMO-81 Rating → Multiplier Conversion

Each categorical rating is mapped to a numeric multiplier. The table below shows the mapping for key drivers:

Driver	vl	l	n	h	vh	xh
RELY	0.75	0.88	1.00	1.15	1.40	1.40
CPLX	0.70	0.85	1.00	1.15	1.30	1.65
TIME	1.00	1.00	1.00	1.11	1.30	1.66
STOR	1.00	1.00	1.00	1.06	1.21	1.56
ACAP	1.46	1.19	1.00	0.86	0.71	0.71
PCAP	1.42	1.17	1.00	0.86	0.70	0.70
TOOL	1.24	1.10	1.00	0.91	0.83	0.83
SCED	1.23	1.08	1.00	1.04	1.10	1.10

Table 1: COCOMO-81 cost driver multiplier values (selected drivers). Full table: Boehm (1981) Table 8-1.

3. Methodology

3.1 COCOMO-81 Baseline Model

COCOMO-81 (CONstructive COSt Model) is a regression-based algorithmic model for estimating software development effort. The basic intermediate formula is:

```
Effort (PM) = a * (KLOC)^b * EAF
EAF = product of all 15 cost driver multipliers
Schedule (months) = 2.5 * (Effort)^0.38
Mode a b Organic 2.4 1.05 Semi-det. 3.0 1.12 Embedded 3.6 1.20
```

The app uses the semidetached mode constants ($a=3.0$, $b=1.12$) to produce a COCOMO baseline for comparison. The EAF (Effort Adjustment Factor) is the product of all 15 multiplier values. An EAF below 1.0 indicates a high-capability / well-equipped team; above 1.0 indicates stress factors.

3.2 Chang (1996) FAHP Feature Weighting

Fuzzy Analytic Hierarchy Process (FAHP) using Chang's (1996) extent analysis method is used to derive the importance weight of each of the 16 input features. These weights are then embedded directly into the SVR kernel function, forcing the model to pay more attention to higher-importance features.

Standard FAHP requires domain experts to fill in a pairwise comparison matrix using Triangular Fuzzy Numbers (TFNs). **In this implementation, the expert judgements are approximated automatically** using the Spearman rank correlation between each feature and a project-level weight vector derived from effort and LOC. This is an indirect proxy and is a known limitation — see Section 7.

3.3 FAHP Pairwise Comparison Matrix

The Chang (1996) algorithm works as follows:

- **Step 1 — TFN Scale:** Integer importance ratios 1–9 are mapped to Triangular Fuzzy Numbers (l , m , u). For example, ratio 3 $\rightarrow$ TFN (1.0, 3.0, 5.0). Ratios between integers are linearly interpolated.
- **Step 2 — Pairwise matrix:** For n features, an $n \times n$ matrix M is built where $M[i][j]$ is the TFN representing how much more important feature i is than feature j . $M[j][i] = \text{reciprocal TFN}$.
- **Step 3 — Synthetic extent values:** For each feature i , compute $S_i = (\text{sum of row } i) \times (\text{inverse of total sum})$. This gives a fuzzy extent value per feature.
- **Step 4 — Degree of possibility:** $V(S_i \geq S_j)$ is computed for all pairs — the degree to which S_i is at least as large as S_j . The minimum over all $j \neq i$ gives the possibility weight W'_i .
- **Step 5 — Normalise:** $W = W' / \text{sum}(W')$ gives the final normalised feature weight vector.
- **Consistency check:** Crisp AHP consistency ratio (CR) is computed on the pairwise matrix. $CR > 0.1$ indicates inconsistency (acceptable threshold in AHP literature is $CR < 0.1$).

3.4 Weighted-Kernel SVR (approximating LSSVM)

The original Sehra et al. (2018) paper uses a Least Squares Support Vector Machine (LSSVM) with a custom weighted RBF kernel based on FAHP weights. The kernel function is:

```
K(x_k, x_l) = exp( -gamma * || sqrt(theta) * (x_k - x_l) ||^2 ) where theta = diag(fahp_w)
is the diagonal weight matrix sqrt(theta) * x pre-scales each feature by sqrt(weight_i)
Equivalent to: X_weighted = X_scaled * sqrt(fahp_w) then use standard RBF: K(X_w_k, X_w_l)
= exp(-gamma * ||X_w_k - X_w_l||^2)
```

In this implementation, sklearn's SVR (inequality-constrained QP) is used instead of the equality-constrained LSSVM from the paper. The weighted kernel is realised by pre-multiplying the scaled feature matrix by sqrt(fahp_w) before fitting SVR — this is mathematically equivalent to embedding the weights in the kernel.

Note: sklearn SVR solves an inequality-constrained QP (Vapnik 1995) whereas LSSVM solves an equality-constrained system (Suykens & Vandewalle 1999). The solutions differ; sklearn SVR is an approximation of the paper's LSSVM.

3.5 Leave-One-Out Cross-Validation (LOO-CV)

LOO-CV is used for two purposes: hyperparameter selection and performance reporting.

- For **hyperparameter search**: a grid of $C \in \{0.5, 1, 5, 10, 50, 100, 200, 500\}$ and $\gamma \in \{0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0, \text{'scale'}\}$ is evaluated. The (C, γ) pair minimising LOO-MMRE is selected. A fine search is then run around the coarse best.
- For **performance reporting**: the LOO-MMRE is the headline metric shown in the app header. It is the mean of $|\text{predicted} - \text{actual}| / \text{actual}$ over all n leave-one-out folds.
- For **prediction interval estimation**: the signed relative errors from LOO (not absolute) are stored. The 10th and 90th percentile of these signed errors give the empirical 80% PI bounds.

3.6 MMRE — Mean Magnitude of Relative Error

MMRE is the standard evaluation metric in software effort estimation research:

```
MRE_i = |predicted_i - actual_i| / actual_i MMRE = (1/n) * sum(MRE_i)
```

MMRE = 0.0 is perfect. A commonly cited acceptable threshold in the literature is $\text{MMRE} < 0.25$. The app displays the LOO-MMRE (which is less optimistic than training MMRE) as the headline statistic.

3.7 Empirical 80% Prediction Interval

Rather than assuming a parametric error distribution, the PI is derived empirically from the LOO signed relative errors:

```
signed_RE_i = (predicted_i - actual_i) / actual_i pi_lo = 10th percentile of signed_RE
(lower bound) pi_hi = 90th percentile of signed_RE (upper bound) For a new estimate E:
PI_low = E * (1 + pi_lo) PI_high = E * (1 + pi_hi)
```

A signed RE of -0.3 means the model under-predicted by 30%; +0.4 means over-predicted by 40%. The 10th–90th percentile range covers 80% of LOO predictions. This PI is displayed in the results card as $[\text{PI}_{\text{low}} \text{ PM}, \text{PI}_{\text{high}} \text{ PM}]$.

4. Training Pipeline

The full training pipeline is implemented in **train_model.py**. It runs once and saves all artefacts to **model.pkl**. The Streamlit app loads model.pkl at startup and never retrains.

Step-by-step pipeline

```
raw X → log(X) → MinMaxScaler → × sqrt(fahp_w) → SVR raw y → log1p(y) → MinMaxScaler → SVR target
```

Step 1 — Load data	Load cocomo81.csv (63 rows). Parse NASA93_DATA inline (93 rows). Concatenate → 156 rows. Drop NaN and zero-feature rows.
Step 2 — FAHP weights	Compute project-level weights W_{effort} and W_{loc} from effort and LOC vectors using FAHP extent analysis. $W_{\text{project}} = 0.5 \cdot W_{\text{effort}} + 0.5 \cdot W_{\text{loc}}$. Compute Spearman correlation of each feature with W_{project} → fahp_w (16-d vector, sums to 1).
Step 3 — Log transform	$X_{\text{log}} = \log(\text{clip}(X_{\text{raw}}, 1e-3, \infty))$. $y_{\text{log}} = \log1p(y_{\text{raw}})$. This compresses the wide LOC/effort range and stabilises training.
Step 4 — MinMaxScaler	$\text{scaler}_X.\text{fit_transform}(X_{\text{log}}) \rightarrow X_{\text{sc}} \in [0,1]^{16}$. $\text{scaler}_y.\text{fit_transform}(y_{\text{log}}) \rightarrow y_{\text{sc}} \in [0,1]$. Scalers are saved to model.pkl.
Step 5 — Kernel weighting	$\text{kernel_w} = \text{sqrt}(\text{fahp_w})$. $X_{\text{w}} = X_{\text{sc}} * \text{kernel_w}$. Features with higher FAHP weight are scaled up, making the RBF kernel more sensitive to them.
Step 6 — Grid search	LOO-MMRE grid over $C \times \text{gamma}$. Coarse grid ($8 \times 8 = 64$ combos) then fine grid around best. Best (C, gamma) minimises LOO-MMRE.
Step 7 — Final model	$\text{SVR}(\text{kernel}='rbf', C=\text{best_C}, \text{gamma}=\text{best_gamma}, \text{epsilon}=0.1).\text{fit}(X_{\text{w}}, y_{\text{sc}})$. Trained on all 156 projects.
Step 8 — LOO signed errors	Re-run LOO with best hyperparameters. Store signed REs → compute pi_lo_pct (10th pct) and pi_hi_pct (90th pct).
Step 9 — Save model.pkl	Pickle dict containing: model, scaler_X, scaler_y, fahp_weights, kernel_w, X_train (weighted), y_train, X_raw, feature_names, loo_mmre, pi_lo_pct, pi_hi_pct, n_cocomo81, n_nasa93, project weights.

Inference (at prediction time)

```
def predict(x_raw): # x_raw: shape (1, 16) x_log = log(clip(x_raw, 1e-3, inf)) # log transform x_sc = scaler_X.transform(x_log) # MinMaxScale x_w = x_sc * kernel_w # FAHP weighting p_sc = model.predict(x_w) # SVR predict p_log = scaler_y.inverse_transform(p_sc) # unscale p_pm = expm1(p_log) # undo log1p return p_pm # person-months
```


5. Application Architecture

5.1 Streamlit App (app.py)

The app is structured as a single-file Streamlit application (~1076 lines). At startup it loads model.pkl via @st.cache_resource to avoid reloading on every interaction. The layout uses a wide Streamlit layout with a fixed dark sidebar (custom CSS) and white content area.

Key sections in app.py

CSS injection	~100 lines of custom CSS injected via st.markdown(unsafe_allow_html=True). Defines dark sidebar (#1a1a2e), blue step-circles, stat pills, card styles, chart containers.
load_model()	@st.cache_resource. Loads model.pkl, validates all required keys. Returns the artifacts dict.
compute_eaf()	Multiplies all 15 cost driver multipliers. Returns EAF scalar.
cocomo_baseline()	Returns effort = $3.0 * loc^{1.12} * eaf$ (semidetached mode).
classify_risk()	Scores effort, EAF, and LOC. $EAF < 0.9$ gives a risk credit (good team). Returns Low/Medium/High with colour code.
whatif()	Iterates improvable cost drivers. For each, simulates rating shift (+1 for capability drivers, -1 for stress drivers). Returns top-5 savings ranked by PM reduction.
run_shap()	@st.cache_data. SHAP KernelExplainer with background=30 random training points, nsamples=150. Returns shap_values array (16,) in log-scaled space.
shap_to_pm()	Converts SHAP log-space values to PM-unit contributions using the scaler_y inverse transform.
Header	Page title + 4 stat pills: LOO-MMRE (blue), n=156 (green), COCOMO-81 (dark), Chang 1996 FAHP (dark).
Sidebar	16 input sliders grouped by category: Size (LOC), Project/Product (RELY/DATA/CPLX), Computer (TIME/STOR/VIRT/TURN), Personnel (ACAP/AEXP/PCAP/VEXP/LEXP), Process (MODP/TOOL/SCED).
Pipeline panel	6 collapsible step cards with blue circle badges, methodology description, and honest implementation caveats.
Results section	Shown after clicking 'Estimate Effort'. Displays: effort PM card, COCOMO baseline, EAF, schedule, PI range, risk badge, executive summary.
SHAP chart	Horizontal bar chart. Blue bars = increases effort, red bars = decreases effort. X-axis in PM units.
Confidence chart	Scatter plot of LOO predicted vs actual across all 156 training projects. Diagonal perfect-prediction line. User's estimate plotted as a red star.
What-If panel	Table of top-5 actionable improvements with projected new estimate and savings PM.
GenAI panel	Three GenAI multiplier scenarios (Conservative 18%, Moderate 26%, Aggressive 45%) applied to the base estimate.

Limitations panel	Honest disclosure of 5 known limitations: data era, SVR approximation, FAHP indirect weights, wide PI, domain specificity.
Footer	Version, PI range, disclaimer.

5.2 Model Training (train_model.py)

train_model.py is run once offline (not at app startup) to produce model.pkl. It takes approximately 5–15 minutes on a standard laptop due to the LOO-CV grid search over 156 projects × 64+ hyperparameter combos. After editing the dataset or methodology, re-run this script and restart the Streamlit app.

```
python train_model.py streamlit run app.py
```

5.3 File Structure

```
Finalthingy/ ■■■ app.py # Streamlit application (~1076 lines) ■■■ train_model.py #
Training script (run once) ■■■ model.pkl # Trained model + all artefacts ■■■
cocomo81.csv # COCOMO-81 dataset (63 projects) ■■■ .streamlit/ ■■■ config.toml #
Streamlit theme (base = 'light')
```

6. App User Guide

6.1 Sidebar — Project Inputs

The left sidebar contains all project inputs. Each input is a slider. Hover over the slider label to see the full cost driver name. Inputs are grouped into 5 categories:

LOC (KLOC)	Lines of code in thousands. Range: 0.5 – 500 KLOC. This is typically the single highest-impact input.
RELY	Required software reliability. Low = occasional inconvenience. Very high = financial/human loss risk.
DATA	Database size relative to program size. Affects data handling complexity.
CPLX	Software complexity. Covers control, computational, device management, and data management operations.
TIME	Execution time constraint as % of available CPU. >50% is high; >95% is extra high.
STOR	Main storage constraint as % of available storage. Similar scale to TIME.
VIRT	Virtual machine volatility. How often the underlying OS/platform changes.
TURN	Computer turnaround time. Relevant for batch environments.
ACAP	Analyst capability. Top 15% = very high; bottom 15% = very low.
AEXP	Applications experience in the problem domain (years).
PCAP	Programmer capability. Similar scale to ACAP.
VEXP	Virtual machine experience (months/years).
LEXP	Programming language experience (months/years).
MODP	Modern programming practices: structured programming, design/code reviews.
TOOL	Software tools usage: from basic to advanced, integrated toolset.
SCED	Schedule constraint. 75% of nominal = very low (accelerated). 160% = very high (relaxed).

6.2 Pipeline Steps Panel

The main content area shows a 6-step pipeline panel explaining the methodology. Each step has a blue numbered circle badge, a title, a description, and an italicised caveat noting any implementation differences from the academic ideal. Click through each step to understand what the model is doing before interpreting results.

Step 1 — FAHP Weighting	Explains how feature weights are derived via FAHP extent analysis and the Spearman correlation proxy.
Step 2 — Log + Scale	Describes the log transform and MinMaxScaler applied to X and y.

Step 3 — Weighted Kernel SVR	Explains the kernel weighting via <code>sqrt(fahp_w)</code> and the SVR vs LSSVM distinction.
Step 4 — LOO-CV Tuning	Describes the grid search and LOO-MMRE evaluation.
Step 5 — SHAP Explainability	Explains KernelExplainer, background samples, and PM-unit conversion.
Step 6 — Empirical PI	Describes how the 80% prediction interval is derived from LOO signed errors.

6.3 Estimate Button & Results

Click the red **"Estimate Effort"** button in the sidebar to run the prediction. Results appear in the main content area:

- **Effort estimate card:** Large predicted effort in PM. Shows prediction interval [PI_low, PI_high] at 80% confidence.
- **COCOMO benchmark:** The COCOMO-81 formula result for comparison. This is a parametric baseline, not the SVR output.
- **EAF:** Product of all 15 multipliers. Values below 1.0 indicate a high-performing team; above 1.0 indicates risk factors.
- **Schedule:** Estimated calendar months using $\text{cal_months} = 2.5 \times \text{effort}^{0.38}$.
- **Risk badge:** Low (green) / Medium (amber) / High (red). Based on effort level, EAF, and LOC.
- **Executive summary:** A plain-English paragraph summarising the estimate, PI, risk, and COCOMO comparison.

6.4 SHAP Explainability Chart

After prediction, a horizontal bar chart shows each feature's contribution to the estimate in person-months. Bars extending right (blue) increase the estimate vs. the baseline; bars extending left (red/negative) decrease it. Features are sorted by absolute contribution magnitude.

The SHAP values are computed using KernelExplainer with 30 background training points (randomly sampled) and 150 perturbation samples. They are computed in the model's log-scaled space and then converted back to PM units using the inverse of `scaler_y`. The conversion is approximate because the log-space SHAP values do not perfectly map to linear PM units, but they give a reliable relative ranking of feature importance.

6.5 Confidence Band Chart

A scatter plot shows LOO-predicted vs. actual effort (PM) for all 156 training projects. The diagonal line represents perfect prediction. Points above the line = over-estimate; below = under-estimate. The user's current input is plotted as a red star at its predicted effort position, giving a sense of where the new project sits relative to the training distribution.

6.6 What-If Optimizer

The What-If panel suggests the top 5 cost driver changes that would most reduce the estimated effort. For each improvable driver, the slider is shifted by one level (+1 for capability drivers like ACAP, PCAP; -1 for stress drivers like CPLX, RELY, TIME) and the new estimate is computed. Results are ranked by PM savings and shown in a table.

The improvable drivers and their directions are:

ACAP, PCAP, AEXP, VEXP, LEXP	Increase (+1 level) — higher capability → lower multiplier → less effort
MODP, TOOL	Increase (+1 level) — better practices/tools → lower multiplier
CPLX, RELY, TIME, STOR	Decrease (–1 level) — lower requirement/constraint → lower multiplier → less effort

6.7 GenAI Productivity Adjustment

A post-hoc GenAI productivity adjustment panel shows how AI-assisted coding tools might reduce the effort estimate. Three scenarios are provided based on published research:

Conservative (–18%)	Peng et al. (2023): GitHub Copilot reduces task completion time by 55% on targeted tasks; 18% overall productivity lift.
Moderate (–26%)	Kalliamvakou (2022), GitHub research: developers complete tasks 26% faster with Copilot assistance.
Aggressive (–45%)	McKinsey (2023): AI coding tools can reduce development time by 20–45% on certain projects.

Note: These multipliers are applied post-hoc to the SVR estimate and are NOT part of the trained model. They represent external research findings and should be interpreted as approximate planning figures.

6.8 Validity & Limitations Panel

The bottom of the results page includes an honest disclosure of 5 known limitations (see also Section 7). This section is important for academic integrity and for communicating the demo's boundaries to research evaluators.

7. Known Issues & Honest Limitations

The following limitations are disclosed both in the app UI (Validity & Limitations panel) and in the pipeline step caveats. They are important for academic integrity when presenting this as a research demo.

7.1 Data Era (1971–1987)

Both datasets (COCOMO-81 and NASA-93) contain projects from 1971 to roughly the mid-1980s. Modern software development practices, toolchains, languages, and team structures differ dramatically. Effort estimates generated by this model should not be applied to modern projects without domain-specific recalibration.

7.2 SVR $\neq$ LSSVM

The original Sehra et al. (2018) paper uses a Least Squares Support Vector Machine (LSSVM, Suykens & Vandewalle 1999), which minimises a regularised least-squares loss with equality constraints. sklearn's SVR minimises an epsilon-insensitive loss with inequality constraints (Vapnik 1995). The mathematical solutions differ, meaning this implementation is an approximation of the paper's approach.

The kernel weighting (embedding $\sqrt{\text{fahp_w}}$ into X before the RBF kernel) is faithful to Equation 17 of the paper. However the loss function and support vector selection differ from LSSVM.

7.3 FAHP Indirect Weights (Not Expert Pairwise)

Standard FAHP requires a domain expert to fill in a pairwise comparison matrix comparing each feature pair. This implementation derives the pairwise ratios automatically from Spearman rank correlations between each feature and a project-level weight vector (itself derived from effort and LOC).

This is a data-driven proxy for expert judgement. The pairwise matrix is therefore not stable across different datasets and does not represent genuine expert opinion. The consistency ratio (CR) check is performed, but a passing CR does not guarantee the weights are meaningful.

7.4 Wide Prediction Interval

The empirical 80% PI derived from LOO signed relative errors is typically very wide (e.g., $[-50\%, +120\%]$). This reflects the genuine difficulty of effort estimation and the high variance in the LOO predictions. The wide PI is honest — effort estimation is an inherently imprecise task — but it limits the practical utility of the point estimate.

7.5 Small, Domain-Specific Training Set

156 projects is a small training set for a machine learning model. The NASA-93 projects are all NASA government projects from the 1970s–1980s; the COCOMO-81 projects are similarly dated. The model will be poorly calibrated for commercial software, web applications, mobile apps, or any modern software type.

7.6 SHAP Approximate Conversion to PM Units

SHAP values are computed in the model's log-scaled output space. The conversion to PM units is approximate: we apply `scaler_y` inverse transform to the expected value plus each SHAP component individually and take the difference. Because the transform is non-linear (`log1p + MinMaxScale`), this decomposition is not exact. The SHAP values should be interpreted as relative importance indicators, not precise PM contributions.

8. Code Bugs Fixed

The following bugs were identified and corrected during development:

Bug 1 — `whatif()` direction inversion

The original `whatif()` function applied rating changes in the wrong direction for capability drivers. Capability drivers like ACAP and PCAP have an *inverse* relationship with effort: a higher capability rating (e.g., h → vh) corresponds to a *lower* COCOMO multiplier (e.g., 0.86 → 0.71), which reduces effort. The original code was shifting these drivers down instead of up.

```
# Fixed: capability drivers use +1 (move up the scale)
improvable = { 'ACAP': ('Increase analyst capability', +1), 'PCAP': ('Increase programmer capability', +1), 'AEXP': ('Increase application experience', +1), # ... other capability drivers: +1
'CPLX': ('Reduce complexity', -1), 'RELY': ('Lower reliability requirement', -1), # ... stress drivers: -1 }
```

Bug 2 — `classify_risk()` EAF handling

The risk classification function was not correctly rewarding a low EAF (high-capability team). A project with $EAF < 0.90$ (meaning all multipliers are below 1 — very capable team, good tools, low complexity) should receive a *risk reduction*, not be treated the same as a nominal EAF. The fix adds a score decrement for $EAF < 0.90$:

```
def classify_risk(effort, eaf, loc):
    score = 0
    if effort > 300: score += 2
    elif effort > 100: score += 1
    if eaf < 0.90: score -= 1 # high capability: risk credit
    elif eaf > 1.15: score += 2
    elif eaf > 1.0: score += 1
    if loc > 100: score += 2
    elif loc > 30: score += 1
    ...
```

Bug 3 — Fabricated NASA-93 data

The original `train_model.py` contained 90 rows of synthetic/fabricated NASA-93 data generated to approximate published statistics. This was replaced with the actual 93-row PROMISE repository dataset parsed directly from the official ARFF file (`timm/ourmine/our/arffs/effest/nasa93.arff`), with categorical ratings converted to COCOMO-81 multipliers using Boehm (1981) Table 8-1.

Impact: the real data has much higher variance (effort 8.4 – 8211 PM vs fabricated 6 – 700 PM; LOC 0.90 – 980 KLOC vs fabricated 3 – 55 KLOC), giving the model a more representative training distribution.

9. References

- [1] Boehm, B. W. (1981). *Software Engineering Economics*. Prentice-Hall, Englewood Cliffs, NJ.
- [2] Chang, D.-Y. (1996). Applications of the extent analysis method on fuzzy AHP. *European Journal of Operational Research*, 95(3), 649–655.
- [3] Sehra, S. K., Brar, Y. S., Kaur, N., & Kaur, S. (2018). Multi-objective optimization for software effort estimation using a fuzzy analytic hierarchy process and least squares support vector machine. *IET Software*, 12(4), 315–323.
- [4] Suykens, J. A. K., & Vandewalle, J. (1999). Least squares support vector machine classifiers. *Neural Processing Letters*, 9(3), 293–300.
- [5] Vapnik, V. (1995). *The Nature of Statistical Learning Theory*. Springer, New York.
- [6] Menzies, T., Hihn, J., & colleagues (2007). NASA-93 dataset. PROMISE Software Engineering Repository. Available: [timm/ourmine](#) on GitHub.
- [7] Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: evidence from GitHub Copilot. *arXiv:2302.06590*.
- [8] Kalliamvakou, E. (2022). Research: Quantifying GitHub Copilot's impact on developer productivity and happiness. GitHub Blog.
- [9] McKinsey & Company (2023). The economic potential of generative AI: the next productivity frontier. McKinsey Global Institute.
- [10] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
- [11] Saaty, T. L. (1980). *The Analytic Hierarchy Process*. McGraw-Hill, New York.
- [12] Jorgensen, M., & Shepperd, M. (2007). A systematic review of software development cost estimation studies. *IEEE Transactions on Software Engineering*, 33(1), 33–53.

Software Effort Estimation Research Demo

Keshav · Mitacs Internship Proposal · 2026

Built on Sehra et al. (2018) · PROMISE NASA-93 · Chang (1996) FAHP · sklearn SVR